Contents

Chapter 1

Making Web Requests with HTTP

The Hypertext Transfer Protocol (HTTP) is the protocol used for transmitting hypertext over the World Wide Web. It is the foundation of any data exchange on the web, and it is a protocol used for transmitting hypertext requests from clients (like a user's browser) to servers, which respond with the requested resources.

1.1 HTTP Requests

An HTTP request is made up of several components:

- **Method**: The HTTP method, like GET (retrieve data), POST (send data), PUT (update data), DELETE (remove data), and so on.
- **URL**: The URL of the resource to retrieve, send data to, update or delete.
- **Headers**: Additional information about the request or response, like the content type of the body.
- **Body**: The body of the request, used when sending data in POST or PUT requests.

One way to think about an HTTP request is to imagine ordering food at a restaurant. The method is the type of action you want to take, such as asking to see the menu, placing an order, changing an order, or canceling it. The URL is like telling the server exactly which restaurant counter to go to. The headers are like the extra details you give, such as saying you want vegetarian food or that you would like water with your meal. The body is the actual order information, such as the list of food you want to buy.

The method tells the server what kind of action the client wants to perform. For example, a GET request is used when the client wants to receive information from the server, such as loading a web page or asking for a list of items. A POST request is used when the client wants to send new information to the server, such as creating an account or submitting a form. A PUT request is used to update something that already exists, and a DELETE request is used to remove something.

The URL works like an address. It tells the request where to go. If the method explains what the client wants to do, the URL explains where that action should happen. For

example, if a web app wants a list of users, it might send a request to a URL that ends in `/users`. If it wants information for one specific user, it might send a request to a URL like `/users/5`.

Headers help the client and server understand each other better. They carry extra information along with the request or response, such as what type of data is being sent. This is similar to writing "Fragile" on a package or adding a note that says "Please handle with care." The message is still the same package, but the note helps explain how it should be handled.

The body is the part of the request that contains the actual data being sent. For example, when a user fills out a sign-up form, the name, email address, and password they enter may be placed in the body of the request. Not every request needs a body. GET requests usually do not send one, because they are mainly asking to receive information rather than send it.

1.2 Using HTTP with Web-Based APIs

Software developers often use HTTP to interact with web-based APIs. An Application Programming Interface (API) is a set of rules that allows programs to talk to each other. The developer creates the API on the server and allows the client to talk to it.

When a developer makes a request to an API endpoint, they're asking the server to either send them some data or receive some data from them. The response from the server will often be in a format like JSON or XML, which the developer can then use in their own application.

For example, a developer might make a GET request to `https://api.example.com/users` to retrieve a list of all users. The server would respond with a list of users in a format like JSON.

An API endpoint is a specific URL that a client can use to communicate with a server. Each endpoint usually has a clear purpose. One endpoint might return a list of books, while another might return information about a single book. In the same way that a restaurant has different counters for ordering food, paying, or picking up an order, an API can have different endpoints for different tasks.

When a client sends a request to an API, the server reads the request and decides how to respond. If the request is valid, the server sends back a response with the requested data or a message saying the action worked. If something is wrong, the server sends back a response explaining that there was a problem. This back-and-forth communication is one of the main ways web applications work behind the scenes.

For example, imagine a school website that stores a list of students. A client could send a GET request to an endpoint like `/students` to ask for the full list. It could send a

POST request to the same endpoint to add a new student. It could send a PUT request to `/students/3` to update the information for one student, or a DELETE request to `/students/3` to remove that student from the list. These requests follow the same HTTP ideas, even if the website or app itself looks very different on the screen.

Many modern apps work this way. A weather app may send a request to get the latest forecast. A music app may send a request to get a list of songs. A shopping app may send a request to place an order. Even though users may only see buttons, menus, and pictures, HTTP requests are often moving data in the background.

1.3 HTTP Responses

After a server receives an HTTP request, it sends back an HTTP response. The response is the server's answer to the client. Just as a request has several parts, a response also has its own important parts.

An HTTP response usually includes a status code, headers, and sometimes a body. The status code tells the client what happened. The headers give extra information about the response. The body contains the returned data, if there is any.

A simple way to think about a response is to imagine asking a librarian for a book. The librarian might say, "Here is the book you asked for," "That book is checked out," or "We do not have that book." These are different kinds of responses to the same kind of request.

Some common HTTP status codes are:

- 200: The request worked, and the server is sending back the requested data.
- 201: The request worked, and the server created something new.
- 400: The request had a problem, such as missing or incorrect information.
- 404: The server could not find what was requested.
- 500: Something went wrong on the server.

These codes help the client understand the result quickly. Instead of reading a long message first, the client can look at the status code and get a fast summary of what happened.

1.4 Data Formats in Responses

When a server sends data back to a client, that data needs to be organized in a way both sides can understand. Two common formats are JSON and XML. These formats are simply ways of arranging information so that programs can read it clearly and consistently.

JSON is one of the most common formats used with web APIs. It is popular because it is lightweight and easy for both humans and programs to read. A JSON response might include data written as key–value pairs. For example, a user might be described with data such as a name, age, and email address.

Here is a small example of JSON:

```
{
  "name": "Ava",
  "grade": 10,
  "club": "Robotics"
}
```

In this example, `"name"`, `"grade"`, and `"club"` are the labels, and `"Ava"`, `10`, and `"Robotics"` are the values. This is similar to filling out an information card. Each label tells what kind of information is being stored, and each value is the actual information.

XML is another format that may be used. It organizes information with opening and closing tags. While it is less common in many modern beginner examples, it is still useful to know that different APIs may return data in different formats.

CHAPTER 2

Using and Creating APIs

As a software engineer, you are likely familiar with building applications that interact with various external services and data sources. One of the most common methods for communication and integration is through HTTP APIs (Application Programming Interfaces). HTTP APIs provide a standardized way for applications to exchange data and functionality over the internet.

This chapter will introduce you to the world of HTTP APIs and explore how you can leverage them in your software development projects. We will cover the fundamental concepts, techniques, and best practices for effectively working with HTTP APIs.

An HTTP API allows two software systems to communicate and exchange information using the Hypertext Transfer Protocol (HTTP). It enables your application to make requests to an API server and receive responses in a structured format, such as JSON (JavaScript Object Notation) or XML (eXtensible Markup Language).

Using HTTP APIs offers a range of benefits for software engineers. It allows you to leverage external services and data sources, enabling your application to access functionality or retrieve valuable information from third-party systems. This opens up opportunities for integration with popular platforms, social media networks, payment gateways, geolocation services, and much more.

Throughout this chapter, we will explore various aspects of working with HTTP APIs, including:

- API endpoints and methods: Understanding how to interact with an API involves identifying the available endpoints and the supported methods, such as GET, POST, PUT, DELETE, and so on. We will discuss how to construct API requests and handle different response formats.
- Authentication and authorization: Many APIs require authentication to ensure secure access and protect sensitive data. We will delve into different authentication mechanisms, including API keys, tokens, OAuth, and other authentication protocols commonly used in API integrations.
- Request parameters and payloads: APIs often accept additional parameters or payloads to customize the request or send data for processing. We will explore how to pass query parameters, request headers, and request bodies when interacting with APIs.
- Error handling and status codes: Learning how to handle errors and interpret status

codes returned by APIs is crucial for building robust and resilient applications. We will discuss common status codes and best practices for handling various scenarios gracefully.

- Rate limiting and throttling: Many APIs impose restrictions on the number of requests you can make within a given timeframe to prevent abuse and ensure fair usage. We will cover techniques for handling rate limiting and implementing efficient strategies to manage API quotas.
- API documentation and testing: Proper documentation is essential for understanding an API's capabilities, endpoints, and expected behavior. We will explore how to read and interpret API documentation, as well as techniques for testing and validating API integrations.

By mastering the art of using HTTP APIs, you will expand your development toolkit and gain the ability to seamlessly integrate your applications with external services, leverage their functionalities, and build powerful, interconnected systems.

So, let's dive into the world of HTTP APIs and uncover the endless possibilities they offer for enhancing your software engineering projects.

FIXME talk about API keys, API requests, how it interacts with HTTP requests, why keys should not be shared

Chapter 3

Data Compression and Decompression

Data compression and decompression are fundamental techniques used in modern computing, enabling efficient storage and transmission of data. The concept of entropy, borrowed from the field of information theory, plays a crucial role in determining the compression rate.

3.1 Data Compression and Decompression

Data compression is the process of reducing the amount of data needed to represent a particular set of information. The two main types of data compression are lossless and lossy. Lossless compression ensures that the original data can be perfectly reconstructed from the compressed data, whereas lossy compression allows some loss of data for more significant compression rates. Decompression is the reverse process of compression, reconstructing the original data from the compressed format.

3.2 Entropy

In information theory, entropy measures the unpredictability or randomness of information content. More specifically, it quantifies the expected value of the information contained in a message. Lower entropy implies less randomness and more repetitiveness, which in turn means the data can be compressed more.

3.3 Entropy and Compression

The role of entropy in data compression is fundamental. The entropy of a source of data is the minimum number of bits required, on average, to encode symbols drawn from the source. It serves as a lower bound on the best possible lossless compression rate.

For a source X with probability distribution $p(x)$, the entropy $H(X)$ is defined as:

$$H(X) = -\sum_{x \in X} p(x) \log_2 p(x) \tag{3.1}$$

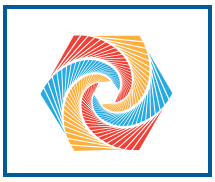
If the entropy of the data is high (i.e., the data is random and unpredictable), the potential for compression is low. On the other hand, if the entropy is low (the data is predictable), the data can be compressed to a smaller size.

Chapter 4

Dealing with JSON and XML

Appendix A

Answers to Exercises



Index